<JUDUL TUGAS AKHIR>

LAPORAN TUGAS AKHIR

Disusun sebagai syarat kelulusan tingkat sarjana

Oleh

MUHAMMAD ADHA RIDWAN

NIM : 13523098



PROGRAM STUDI SARJANA TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
<Bulan Tahun>

<JUDUL TUGAS AKHIR>

LAPORAN TUGAS AKHIR

Oleh

MUHAMMAD ADHA RIDWAN
NIM : 13523098

Program Studi Sarjana Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Telah disetujui dan disahkan sebagai Laporan Tugas Akhir
di Bandung, pada tanggal <Tanggal Pengesahan>

Pembimbing I,	Pembimbing II,
Ir. Yudistira Dwi Wardhana Asnar Ph.D	Galuh Barata ST MT
NIP. <NIP Pembimbing I>	NIP. <NIP Pembimbing II>

LEMBAR PERNYATAAN

Dengan ini saya menyatakan bahwa:

1. Pengerjaan dan penulisan Laporan Tugas Akhir ini dilakukan tanpa menggunakan bantuan yang tidak dibenarkan.
2. Segala bentuk kutipan dan acuan terhadap tulisan orang lain yang digunakan di dalam penyusunan laporan tugas akhir ini telah dituliskan dengan baik dan benar.
3. Laporan Tugas Akhir ini belum pernah diajukan pada program pendidikan di perguruan tinggi mana pun.

Jika terbukti melanggar hal-hal di atas, saya bersedia dikenakan sanksi sesuai dengan Pelanggaran Peraturan Akademik dan Kemahasiswaan Institut Teknologi Bandung.

Bandung, <Tanggal Pengesahan>

Muhammad Adha Ridwan

NIM : 13523098

ABSTRAK

<JUDUL TUGAS AKHIR>

Oleh

MUHAMMAD ADHA RIDWAN

NIM : 13523098

Tuliskan ringkasan abstrak bahasa Indonesia di sini. Abstrak memuat permasalahan yang dikaji, metode/teknik keinformatikaan yang digunakan untuk menyelesaikan masalah, hasil yang dicapai, dan kesimpulan. Panjang abstrak maksimal 300 kata dan ditulis dalam satu spasi.

Kata Kunci: Kata Kunci 1, Kata Kunci 2, Kata Kunci 3

ABSTRACT

<ENGLISH THESIS TITLE>

By

MUHAMMAD ADHA RIDWAN

NIM : 13523098

Write the English abstract here. It summarizes the research problem, methodology, key findings, and conclusions. Maximum 300 words, single-spaced.

Keywords: Keyword 1, Keyword 2, Keyword 3

KATA PENGANTAR

Puji dan syukur dipanjatkan ke hadirat Tuhan Yang Maha Esa atas berkat dan rahmat-Nya sehingga Laporan Tugas Akhir ini dapat diselesaikan. Penulis mengucapkan terima kasih yang sebesar-besarnya kepada:

1. Dosen Pembimbing yang telah memberikan bimbingan, arahan, dan masukan berharga.
2. Para dosen dan staf pengajar Program Studi Sarjana Teknik Informatika STEI ITB.
3. Rekan-rekan mahasiswa dan seluruh pihak yang telah mendukung pelaksanaan tugas akhir ini.

DAFTAR ISI

BAB I Pendahuluan 1

1.1 Latar Belakang 1

1.2 Rumusan Masalah 1

1.3 Tujuan Tugas Akhir 1

1.4 Batasan Masalah 1

1.5 Metodologi 1

1.6 Sistematika Pembahasan 1

BAB II Tinjauan Pustaka 3

2.1 Keamanan Kode Hasil LLM 3

2.1.1 Keamanan sebagai dimensi yang berbeda dari kebenaran fungsional 3

2.1.2 Prompt sintetis, interaksi nyata, dan validitas pengukuran 4

2.1.3 Dataset dan alasan pemilihan 5

2.2 Deteksi Kerentanan melalui SAST dan Pendekatan Hibrida .6

2.2.1 Cakupan dan ketidakpastian analisis statis 6

2.2.2 Pengayaan spesifikasi dan triase kontekstual 7

2.2.3 SAST sebagai jangkar keputusan 8

2.3 Sistem Multi-Agen dan Agregasi Keputusan Keamanan ... 10

2.3.1 Tujuan kolaborasi dan pembagian tanggung jawab . 10

2.3.2 Integrasi generasi, deteksi, dan pengujian 10

2.3.3 Voting, debat, dan penilaian berbasis peran 11

2.3.4 Arbiter, biaya, dan batas keputusan 12

2.4 Perbaikan Otomatis dan Pengendalian Regresi 14

2.4.1 Umpan balik eksternal untuk remediasi 14

2.4.2 Pemisahan Fixer dan penggunaan pengetahuan 15

2.4.3 Regresi, pemindaian ulang, dan penghentian 15
2.5 Ketahanan Gate terhadap Manipulasi Reviewer 17
2.5.1 Permukaan ancaman dan perbedaan sasaran serangan 17
2.5.2 Bukti yang berbeda tentang kerentanan penilai 17
2.5.3 Sanitizer dan batas kebijakan sebagai hipotesis pertahanan 18
2.6 Ringkasan Penelitian Terdahulu dan Celah Penelitian 19
2.6.1 Perbandingan cakupan dan bukti 19
2.6.2 Celah integrasi, pertahanan, dan pengukuran 21
BAB III Analisis dan Perancangan Solusi 24
3.1 Analisis Masalah 24
3.2 Kebutuhan Sistem 24
3.3 Perancangan Arsitektur 24
BAB IV Implementasi dan Evaluasi 25
4.1 Lingkungan Implementasi 25
4.2 Prosedur dan Skenario Pengujian 25
4.3 Hasil Pengujian dan Diskusi 25
BAB V Kesimpulan dan Saran 26
5.1 Kesimpulan 26
5.2 Saran 26
DAFTAR PUSTAKA 27
LAMPIRAN A Pernyataan Penggunaan Kecerdasan Buatan (AI) 28
LAMPIRAN B Contoh Lampiran Teknis 30
B.1 Spesifikasi Lingkungan Pengujian 30

DAFTAR GAMBAR

DAFTAR TABEL

Tabel 1 Daftar Singkatan yang Digunakan dalam Laporan x
Tabel 2 Perbandingan penelitian terdahulu untuk multi-agent security gate . . 20
Tabel 3 Rincian Penggunaan Alat AI dalam Penyusunan Tugas Akhir 28

DAFTAR SINGKATAN DAN ISTILAH

Tabel 1: Daftar Singkatan yang Digunakan dalam Laporan

Singkatan	Keterangan
AI	Artificial Intelligence (Kecerdasan Buatan)
API	Application Programming Interface
ITB	Institut Teknologi Bandung
STEI	Sekolah Teknik Elektro dan Informatika
TA	Tugas Akhir

BAB I
PENDAHULUAN

1.1 Latar Belakang

Tuliskan latar belakang masalah di sini. Berisi dasar pemikiran, kebutuhan, atau alasan yang menjadi ide dari topik tugas akhir di bidang Teknik Informatika.

1.2 Rumusan Masalah

Rumusan masalah menguraikan permasalahan spesifik yang diselesaikan, yang mencakup:

1. Kondisi atau situasi yang ada saat ini terkait bidang kajian.
2. Pokok persoalan utama beserta kelemahan atau keterbatasannya.
3. Alasan pentingnya menyelesaikan pokok persoalan tersebut.
4. Usulan pendekatan solusi secara ringkas.

1.3 Tujuan Tugas Akhir

Tujuan tugas akhir merumuskan target yang ingin dicapai yang dapat diukur dan diuji pada tahap evaluasi.

1.4 Batasan Masalah

Batasan masalah mendefinisikan ruang lingkup pengerjaan tugas akhir agar fokus dan terarah.

1.5 Metodologi

Metodologi menguraikan tahapan sistematis yang dilakukan selama pengerjaan tugas akhir keinformatikaan, mulai dari analisis kebutuhan, perancangan, implementasi, hingga pengujian.

1.6 Sistematika Pembahasan

Sistematika pembahasan memberikan ringkasan singkat isi laporan per bab:

- **BAB I Pendahuluan**: Menguraikan latar belakang, rumusan masalah, tujuan, batasan, metodologi, dan sistematika pembahasan.
- **BAB II Studi Literatur**: Membahas teori dasar dan penelitian terkait.
- **BAB III Analisis dan Perancangan Solusi**: Menguraikan analisis kebutuhan dan desain arsitektur solusi.
- **BAB IV Implementasi dan Evaluasi**: Mendokumentasikan hasil pengujian dan analisis perbandingan.
- **BAB V Kesimpulan dan Saran**: Menyimpulkan hasil ketercapaian tujuan dan saran pengembangan lanjutan.

BAB II
TINJAUAN PUSTAKA

Bab ini meninjau dasar empiris dan pendekatan teknis untuk memeriksa keamanan kode hasil *large language model* (LLM). Pembahasan disusun berdasarkan enam tema: keamanan keluaran model, deteksi hibrida, koordinasi multi-agen, remediasi, ketahanan penilai, serta posisi penelitian. Perbandingan dibatasi pada konteks eksperimen masing-masing sumber. Rancangan *security gate* yang dibahas merupakan usulan penelitian, sedangkan konfigurasi model, jumlah iterasi, dan cakupan pengujian dinamis masih perlu ditentukan. Tanda [VERIFIKASI Vxx] menunjukkan pekerjaan yang harus diselesaikan sebelum naskah difinalkan.

2.1 Keamanan Kode Hasil LLM

2.1.1 Keamanan sebagai dimensi yang berbeda dari kebenaran fungsional

Kemampuan menghasilkan program yang dapat dijalankan tidak dengan sendirinya menunjukkan bahwa program tersebut memenuhi kebutuhan keamanan. Implementasi dapat memberikan keluaran yang benar pada masukan normal, tetapi gagal membatasi masukan penyerang, melindungi informasi sensitif, atau menegakkan otorisasi. Dalam evaluasi kode LLM, pembedaan ini menentukan apa yang sebenarnya diukur: keberhasilan menyelesaikan tugas, keberadaan pola kelemahan, atau kemungkinan eksploitasi. *Common Weakness Enumeration* (CWE) digunakan oleh dataset yang ditinjau untuk mengelompokkan jenis kelemahan, sehingga perbandingan perlu mempertahankan kategori dan unit analisisnya [Bhatt et al., 2023; Siddiq dan Santos, 2022; Tony et al., 2023]. Tidak ditemukannya CWE tertentu oleh suatu alat hanya merupakan hasil pemeriksaan dengan cakupan alat tersebut.

Pada CyberSecEval, pengujian keluarga Llama 2, Code Llama, dan GPT mencakup kecenderungan menghasilkan kode tidak aman serta kepatuhan

terhadap permintaan bantuan serangan. Rata-rata saran kode rentan dilaporkan sekitar 30% pada kasus uji benchmark tersebut. Detektor kode tidak amannya memperoleh presisi 96% dan *recall* 79% dalam validasi manual yang dilaporkan penulis [Bhatt et al., 2023]. Kedua jenis angka itu berbeda: angka pertama menggambarkan keluaran model pada benchmark, sedangkan pasangan presisi dan recall menggambarkan alat ukurnya. Kecenderungan model yang lebih cakap dalam pemrograman untuk tetap menghasilkan pola tidak aman juga mengingatkan bahwa kemampuan coding tidak dapat dipakai sebagai pengganti evaluasi keamanan.

Gambaran serupa muncul pada pengujian tujuh alat oleh Morkonda et al. yang memakai 81 prompt Python LLMSecEval untuk 12 kategori CWE. Alat yang diuji meliputi GPT-4o, Claude 3 Sonnet, Gemini, Perplexity AI, Phind-70b, CodeWhisperer, dan watsonx. Dengan penilaian CodeQL serta GPT-4o untuk kategori tertentu, sedikitnya sekitar 73% cuplikan pada setiap alat mengandung kerentanan [Morkonda et al., 2026]. Angka ini berlaku pada kumpulan tugas berorientasi keamanan yang sengaja dipilih, bukan pada seluruh kode yang dihasilkan LLM. Penggunaan LLM sebagai bagian penilai juga membuat konfirmasi temuan dan independensi evaluasi perlu diperhatikan.

2.1.2 Prompt sintetis, interaksi nyata, dan validitas pengukuran

Eksperimen terkendali memudahkan perbandingan model karena instruksi dan kategori kelemahan dapat diseragamkan. Namun, prompt semacam itu belum tentu merepresentasikan konteks pengembang. Analisis DevGPT menggunakan 2.315 cuplikan C, C++, dan C# dari interaksi nyata, dengan pemindaian statis serta pemeriksaan manual yang mengonfirmasi 56 kerentanan pada 48 berkas. Versi jurnalnya juga menguji deteksi dan perbaikan menggunakan GPT-4.1, GPT-5, serta Claude Opus 4.1 [Belozerov et al., 2026]. Berbeda dari eksperimen prompt keamanan terpilih, unit yang diperiksa di sini mencakup percakapan dan kode yang dibawa pengguna. Proporsi temuannya tidak layak dibandingkan langsung dengan persentase pada LLMSecEval.

WildCode Revisited memperkuat kebutuhan meninjau penggunaan nyata dan perhatian pengguna terhadap keamanan [Khanmohammadi et al., 2026]. Sementara itu, studi lintas metode prompting melaporkan bahwa perubahan prompt dapat menggeser komposisi kategori CWE tanpa memberikan penurunan frekuensi atau kepadatan kerentanan yang signifikan secara statistik pada konfigurasi yang diuji [Kharma et al., 2026]. Dengan demikian, instruksi untuk menulis kode aman perlu diperlakukan sebagai satu kondisi eksperimen, bukan jaminan mitigasi. Temuan pada kode C/C++ dalam LLM-CSEC juga menjadi konteks pelengkap, tetapi karakteristiknya tidak langsung mewakili Python [Shahid et al., 2025].

Kritik terhadap CyberSecEval menunjukkan persoalan lain: komentar, pengenal, dan instruksi yang mengisyaratkan praktik tidak aman dapat memengaruhi hasil benchmark. Perubahan isyarat semacam itu mengubah proporsi keluaran yang ditandai aman oleh detektor [Hariharan et al., 2024]. Artinya, benchmark dapat sekaligus mengukur kepatuhan terhadap instruksi bermasalah dan kecenderungan mereproduksi pola kelemahan. Dataset publik juga berpotensi muncul dalam data pelatihan. Potensi kontaminasi tersebut merupakan ancaman validitas yang perlu diuji atau dibatasi, bukan bukti bahwa model tertentu telah menghafal seluruh dataset [Chen et al., 2025].

2.1.3 Dataset dan alasan pemilihan

Label tugas tidak otomatis mencakup seluruh kelemahan yang mungkin muncul pada keluarannya. Satu prompt bertarget CWE tertentu dapat menghasilkan kelemahan tambahan. Oleh karena itu, label asal dipakai sebagai konteks seleksi, sedangkan penetapan temuan aktual memerlukan pemeriksaan tersendiri.

SecurityEval menyediakan prompt dan contoh kode Python tidak aman dengan label CWE. Penghitungan `dataset.jsonl` pada commit `6f4fb70` menghasilkan 121 sampel dan 69 kategori berdasarkan awalan CWE pada kolom `ID`. LLMSecEval pada commit `39e97f0` berisi 150 prompt, mencakup 18 kategori CWE; berdasarkan kolom `Language`, 83 di antaranya berlabel Python dan

mencakup 12 kategori. Kedua snapshot tersebut memiliki 12 kategori CWE Python yang beririsan [Siddiq dan Santos, 2022; Tony et al., 2023]. Angka ini merupakan pemeriksaan snapshot repositori, bukan jumlah sampel eksperimen yang telah diputuskan. Perbedaan dengan subset 81 prompt pada Morkonda et al. tetap memerlukan penjelasan seleksi [VERIFIKASI V01; VERIFIKASI V02; VERIFIKASI V03].

Kombinasi keduanya relevan karena menyediakan dua bentuk masukan: pelengkapan kode dan instruksi bahasa alami. Untuk rencana lima sampai delapan CWE Python, pemilihan akhir perlu mempertimbangkan jumlah kasus, dukungan Semgrep/Bandit, dan ketersediaan pembuktian dinamis. Duplikasi lintas dataset harus diperiksa sebelum ukuran sampel dijumlahkan. CodeSecEval dapat menjadi pelengkap remediasi, tetapi versinya harus dibedakan: v2 tahun 2024 melaporkan 180 sampel dan 44 jenis kerentanan, sedangkan v3 September 2026 melaporkan 255 tugas Python dengan uji fungsional dan keamanan serta 77 CWE [J. Wang et al., 2024; J. Wang et al., 2026]. Pemakaian versi terbaru membutuhkan pemeriksaan artefak uji dan lisensinya [VERIFIKASI V04].

Sintesis dan Keterbatasan. Bukti dari benchmark dan interaksi nyata sama-sama mendukung pemeriksaan keamanan terpisah dari generasi kode. Namun, variasi bahasa, prompt, model, label, dan alat penilai membatasi generalisasi setiap persentase. Penelitian ini karena itu diarahkan pada evaluasi terkontrol atas subset Python yang dibekukan versinya, dengan label serta pemeriksaan independen yang dapat ditelusuri. Kontribusinya dinilai melalui perubahan risiko pada kondisi tersebut, bukan melalui klaim bahwa keluaran LLM secara umum telah menjadi aman.

2.2 Deteksi Kerentanan melalui SAST dan Pendekatan Hibrida

2.2.1 Cakupan dan ketidakpastian analisis statis

Static application security testing (SAST) memeriksa kode tanpa menjalankan program. Pendekatannya dapat mencakup pencocokan aturan, pemeriksaan struktur sintaksis, dan pelacakan aliran data. Karena itu, SAST tidak tepat digambarkan hanya sebagai pencarian teks. Kelemahannya muncul

ketika model analisis dan aturan tidak cukup merepresentasikan perilaku pustaka, sanitasi khusus aplikasi, atau hubungan antarprosedur. Temuan alat dapat menjadi *false positive*, sedangkan alur yang tidak terjangkau analisis dapat menjadi *false negative*. Literatur hibrida menempatkan LLM pada celah antara peringatan sintaktis dan penafsiran konteks tersebut [Z. Li et al., 2025; Iranmanesh et al., 2025].

Perbedaan cakupan alat juga membuat kesepakatan antarpemindai tidak otomatis menjadi kebenaran acuan. Kode yang tidak ditandai Bandit masih mungkin ditandai CodeQL, sebagaimana dibahas dalam evaluasi Multi-LLMSecCodeEval [Sabir et al., 2026]. Dalam rancangan penelitian ini, penggunaan Semgrep dan Bandit perlu disertai pemetaan aturan ke CWE serta pencatatan versi. Keluaran kosong, kegagalan parsing, dan aturan yang tidak berlaku merupakan keadaan yang berbeda. Ketiganya tidak boleh dilebur menjadi kesimpulan bahwa program bebas kerentanan. Sifat hasil yang relatif berulang pada konfigurasi tetap membantu audit, tetapi tidak menghilangkan keterbatasan cakupan.

2.2.2 Pengayaan spesifikasi dan triase kontekstual

Pendekatan *neuro-symbolic* memanfaatkan LLM untuk mendukung analisis formal atau berbasis aturan. IRIS, misalnya, memakai penalaran LLM untuk membantu spesifikasi *taint* dan analisis kontekstual sebelum menggabungkannya dengan CodeQL. Versi ICLR 2025 melaporkan 55 kerentanan terdeteksi dengan GPT-4 pada CWE-Bench-Java yang berisi 120 kerentanan nyata, dibandingkan 27 dengan CodeQL; rata-rata *false discovery rate* turun dari 90,03% menjadi 84,82% [Z. Li et al., 2025]. Hasil ini menunjukkan manfaat pengayaan analisis, sekaligus memperlihatkan bahwa penurunan tingkat kesalahan tidak berarti semua alarm telah akurat. Angka 69 dari versi lebih awal tidak digunakan dalam perbandingan ini.

Triase menggunakan peran yang lebih sempit: LLM menerima kandidat temuan dan memeriksa apakah konteks mendukungnya. ZeroFalse memperkaya keluaran CodeQL dengan jejak aliran, bukti kontekstual, dan pengetahuan CWE

sebelum penilaian model. Evaluasinya mencakup OWASP Java Benchmark serta proyek nyata, sehingga manfaat prompt khusus CWE tetap terikat pada objek dan konfigurasi tersebut [Iranmanesh et al., 2025]. SAST-Genius juga menggabungkan Semgrep dengan analisis LLM dan melaporkan pengurangan false positive dalam pengujian penulis [Agrawal dan Ahi, 2025]. Namun, rincian benchmark, model triase, dan replikasi hasil tersebut belum cukup diaudit untuk menjadikannya perkiraan peningkatan pada Python [VERIFIKASI V06].

Pembanding paling dekat untuk kombinasi triase dan validasi adalah QASecClaw. Semgrep menghasilkan kandidat, kemudian agen filter berbasis Qwen 3.5 Plus menilai konteksnya. Pada 2.740 kasus Java OWASP Benchmark v1.2, false positive turun dari 560 menjadi 64, dengan recall yang juga sedikit menurun. Jalur korelasi bukti dinamis terdapat dalam arsitekturnya, tetapi tidak aktif pada evaluasi benchmark statis tersebut [Ameen et al., 2026]. Pembedaan antara komponen yang dirancang dan komponen yang benar-benar diuji ini penting: keberadaan agen validasi tidak membuktikan bahwa temuan telah dikonfirmasi melalui eksploitasi. Demikian pula, penurunan alarm harus dinilai bersama kerentanan benar yang ikut tersaring.

QRS memperluas penggunaan LLM dari penyaring alarm menjadi pembentuk kueri CodeQL serta penalaran atas temuan pada paket Python utuh [Tsigkourakos dan Patsakis, 2026]. Perbedaannya dengan eksperimen cuplikan kode terletak pada tersedianya konteks proyek dan lingkungan eksekusi. Secara keseluruhan, IRIS, ZeroFalse, QASecClaw, dan QRS memberi bukti tentang pengayaan analisis atau pengurangan alarm, tetapi objeknya terutama proyek, paket, atau benchmark yang telah tersedia. Hasilnya belum langsung mengukur keamanan kode baru yang dihasilkan generator LLM. Pemindahan pendekatan tersebut ke gate generasi kode merupakan pertanyaan empiris tersendiri.

2.2.3 SAST sebagai jangkar keputusan

Penggabungan hasil memerlukan format bukti yang konsisten. Identitas artefak dan versi patch harus menyertai setiap laporan agar alasan dari pemeriksaan lama tidak digunakan untuk menerima kode yang telah berubah.

Respons model yang tidak dapat dibaca atau tidak lengkap juga tidak boleh ditafsirkan sebagai pembatalan alarm.

Dalam rancangan gate, agen Analyse diusulkan menggabungkan pemeriksaan model dengan keluaran alat, sedangkan Triage menilai relevansi temuan. Pemisahan ini membantu membedakan pencarian kelemahan dari keputusan untuk mempertahankan alarm. Setiap temuan perlu membawa lokasi, CWE, aturan asal, tingkat keparahan, alasan triase, dan bukti pendukung. Deduplicasi harus mempertahankan hubungan antaralarm yang merujuk akar masalah sama, sehingga satu kerentanan tidak dihitung beberapa kali hanya karena dilaporkan oleh beberapa agen. Penjelasan LLM merupakan hipotesis analisis, bukan bukti eksekusi.

Kebijakan yang direncanakan tidak mengizinkan Triage sendirian membatalkan temuan SAST berkeparahan tinggi. Akan tetapi, kebijakan konservatif tersebut dapat meningkatkan penolakan terhadap kode yang sebenarnya benar. Efeknya perlu dilaporkan sebagai pertukaran antara risiko meloloskan kerentanan dan beban revisi atau tinjauan manusia. Penetapan keparahan juga tidak boleh semata-mata menyalin nama CWE; sumber tingkat keparahan dan konteks dampaknya harus dicatat [VERIFIKASI V07]. Jika bukti belum cukup, status temuan tetap tidak pasti dan diserahkan kepada arbiter, bukan dihapus dari catatan.

Sintesis dan Keterbatasan. SAST dan LLM menawarkan bentuk bukti yang saling melengkapi, tetapi keduanya memiliki titik buta. Studi hibrida mendukung triase kontekstual tanpa memberikan jaminan bahwa recall selalu terjaga pada bahasa dan dataset lain. Posisi penelitian ini adalah menguji kombinasi keduanya pada kode hasil LLM, mempertahankan alarm kritis sebagai batas kebijakan, serta mengukur dampak penyaringan terhadap presisi dan recall. Dengan demikian, peran Triage dibenarkan oleh kebutuhan pengelolaan ketidakpastian, bukan oleh anggapan bahwa semua peringatan statis berlebihan.

2.3 Sistem Multi-Agen dan Agregasi Keputusan Keamanan

2.3.1 Tujuan kolaborasi dan pembagian tanggung jawab

Sistem multi-agen membagi pekerjaan ke dalam beberapa peran yang bertukar informasi. Dalam keamanan kode, pembagiannya perlu dibaca bersama tujuan akhir: menghasilkan implementasi, mengklasifikasikan kerentanan pada kode yang tersedia, atau meninjau perubahan. Ketiga tugas tersebut memerlukan konteks dan keluaran berbeda. CodeAgent menempatkan koordinasi serta pengawasan QA dalam proses review perangkat lunak, sedangkan VulTrial dan MAVUL terutama menekankan kualitas penilaian kerentanan [Tang et al., 2024; Widyasari et al., 2025; Y. Li et al., 2025]. Keberhasilan pada salah satu tugas tidak langsung menunjukkan keberhasilan gate yang memutuskan penerimaan kode dan mengelola revisinya.

Pembagian peran juga berbeda dari keberagaman model. Beberapa agen dapat memakai model yang sama dengan instruksi berbeda, sedangkan sistem heterogen memakai model dari keluarga atau ukuran berbeda. Peran yang berbeda dapat memperluas pertanyaan analisis, tetapi keluaran agen tetap mungkin berkorelasi karena latar pelatihan, konteks, dan instruksi serupa. Karena itu, jumlah agen bukan ukuran independensi bukti. Rancangan evaluasi harus menguji apakah penambahan peran atau pergantian model benar-benar mengubah keputusan yang salah, bukan sekadar menambah variasi penjelasan. Biaya koordinasi dan pengulangan analisis merupakan bagian dari konsekuensi rancangan tersebut.

2.3.2 Integrasi generasi, deteksi, dan pengujian

AutoSafeCoder merupakan pembanding langsung karena menghubungkan Coding Agent, Static Analyzer Agent, dan Fuzzing Agent secara iteratif. Pada metode v1, Static Analyzer Agent adalah reviewer LLM yang diarahkan menggunakan CWE; Bandit dipakai untuk menilai kerentanan pada eksperimen, sehingga keduanya tidak boleh disamakan. Dengan GPT-4o, jumlah keluaran rentan pada SecurityEval turun dari 59/121 menjadi 44/121, selisih sekitar 12,4 poin persentase berdasarkan hitungan sampel. Pelaporan penurunan

13% dalam paper tidak ditafsirkan sebagai penurunan relatif. Pada HumanEval, skor pass@1 berubah dari 0,9085 menjadi 0,8780 [Nunez et al., 2024]. Hasil tersebut mendukung manfaat keamanan dengan penurunan fungsionalitas yang perlu diakui. Pengujian fuzzing-nya memeriksa crash, sehingga tidak setiap kelas kelemahan memperoleh bukti eksploitasi yang setara.

Multi-LLMSecCodeEval melangkah lebih jauh dengan membandingkan sepuluh pipeline yang mencakup generasi, deteksi, dan patching. Pada SecLLMEval serta SecLLMHolmes, kombinasi ensembel, CodeQL, dan verifikasi lintas-model dilaporkan lebih konsisten daripada kolaborasi LLM tanpa alat statis [Sabir et al., 2026]. Oleh sebab itu, alur generate–review–repair sendiri tidak dapat dijadikan pembeda penelitian ini. Angka peningkatan rinci belum dipakai karena hasil antarbagian paper perlu direkonsiliasi, termasuk perbedaan pelaporan konfigurasi dan keluaran tersisa [VERIFIKASI V05]. Temuan kualitatifnya tetap relevan untuk menilai kontribusi orkestrasi dibanding sekadar penggunaan model yang lebih besar.

Arsitektur tersebut menunjukkan bahwa bukti dari alat dan interaksi antaragen perlu dibedakan dalam analisis kontribusi. Jika sebuah sistem membaik setelah menambahkan pemindai dan beberapa reviewer sekaligus, peningkatan tidak dapat seluruhnya dikaitkan dengan komunikasi agen. Perbandingan terhadap LLM tunggal, SAST saja, dan SAST ditambah LLM diperlukan untuk memisahkan pengaruh alat dari pengaruh kolaborasi. Dalam penelitian ini, kondisi pembanding diusulkan memakai tugas, anggaran, serta kriteria penilaian yang sebanding. Perbedaan anggaran harus dilaporkan agar sistem yang lebih mahal tidak tampak unggul tanpa konteks.

2.3.3 Voting, debat, dan penilaian berbasis peran

Voting mayoritas menyederhanakan agregasi menjadi jumlah pendapat. Kelemahannya adalah pendapat salah yang berkorelasi dapat mendominasi. ReasonVul menggunakan analisis deduktif, induktif, dan abduktif sebelum debat ketika terdapat konflik. Konfigurasi penulis memasangkan Phi4, Llama4, dan DeepSeek untuk ketiga peran tersebut. Pada analisis konflik eksperimen

PrimeVul, 389 dari 542 konflik diselesaikan dengan benar; generalisasi pada JITVUL dilaporkan dengan PairAcc 28,67% [Peng et al., 2026]. PairAcc menuntut pengenalan pasangan kode rentan dan versi perbaikannya secara benar, sehingga tidak setara dengan akurasi klasifikasi biasa. Keuntungan debat tersebut merupakan hasil pada protokol tertentu, bukan alasan menambah ronde tanpa batas.

Pendekatan ruang sidang memisahkan pihak yang mendukung dan menentang dugaan kerentanan dari pihak yang menilai argumentasinya. VulTrial menggunakan peran peneliti keamanan, penulis kode, moderator, dan review board, serta mengeksplorasi penyesuaian instruksi untuk peran tersebut [Widyasari et al., 2025]. MAVUL menekankan konteks lintas-prosedur dan penyempurnaan interaktif [Y. Li et al., 2025]. Keduanya membantu menjelaskan mengapa temuan perlu dipertanyakan sebelum diterima. Namun, keberadaan hakim atau agen evaluasi tidak otomatis berarti terdapat arbiter operasional yang mengatur penerimaan artefak, batas revisi, dan pelimpahan kepada manusia. Fungsi masing-masing peran harus diperiksa dari implementasinya.

Debat juga dapat memperkuat kesalahan bersama. Refute-or-Promote telah mengusulkan review bertahap, kritik lintas-model, dan gerbang bukti empiris; laporan kasusnya memperlihatkan bahwa kesepakatan reviewer dapat dibantah oleh pengujian nyata [Agarwal, 2026]. Dengan demikian, klaim bahwa literatur tidak memiliki mekanisme gerbang atau penilai berlapis tidak dapat dipertahankan. Pertanyaan yang lebih tepat adalah bagaimana aturan penerimaan dihubungkan dengan bukti, bagaimana kegagalan ditangani, dan apakah rancangan tersebut diuji pada kode hasil LLM serta serangan terhadap reviewer. Uraian kegagalan debat dari sumber yang hanya disebut judul belum digunakan sebagai bukti [VERIFIKASI V08].

2.3.4 Arbiter, biaya, dan batas keputusan

Perbedaan antara agen yang mengemukakan pendapat dan komponen yang berwenang mengambil keputusan perlu dinyatakan dalam kontrak pertukaran informasi. Reviewer dapat menyampaikan tingkat keyakinan, tetapi

arbiter tidak harus memperlakukannya sebagai probabilitas yang terkalibrasi. Bukti uji yang dapat direproduksi, lokasi masalah, dan status eksekusi memiliki fungsi berbeda dari keyakinan tersebut. Untuk audit, catatan keputusan perlu mempertahankan kondisi aturan yang terpenuhi dan alasan kasus dipindahkan ke revisi atau peninjauan manusia. Dengan pemisahan ini, kegagalan proses dapat ditelusuri ke temuan, interpretasi, atau penerapan kebijakan, bukan dilebur menjadi kesalahan model secara umum.

Model heterogen dapat membagi tugas antara pakar yang lebih mahal dan pemeriksa yang lebih ringan. Arsitektur Wang mengombinasikan pakar berbasis layanan dengan verifier lokal untuk mengevaluasi pertukaran kualitas dan biaya [Wang, 2026]. Implikasinya bagi gate adalah biaya tidak cukup dihitung dari satu panggilan generator. Triase, pengulangan debat, pembuatan pengujian, patching, dan kegagalan API dapat menambah konsumsi token serta waktu. Eksekusi paralel juga tidak menghilangkan biaya, meskipun dapat mengurangi waktu tunggu. Harga historis suatu paper tidak dipakai sebagai perkiraan biaya layanan pada penelitian ini.

Arbiter yang diusulkan merupakan komponen non-LLM dengan aturan dalam YAML. Agen menyumbangkan temuan dan interpretasi, sedangkan arbiter menentukan *pass*, *revise*, atau *block*. Temuan berkeparahan tinggi yang belum terselesaikan mencegah pass; revise mengirimkan artefak kepada Fixer; block mengarahkan kasus ke peninjauan manusia. Batas iterasi serta kondisi kegagalan harus eksplisit. Label *Verified* dimaknai sebagai lolos pemeriksaan yang ditentukan, bukan bukti keamanan menyeluruh. Format YAML sendiri bukan kebaruan atau pertahanan; isi aturan, otoritas perubahan, dan jejak keputusanlah yang perlu dinilai [VERIFIKASI V07; VERIFIKASI V09].

Sintesis dan Keterbatasan. Literatur telah mencakup kolaborasi lintas-peran, lintas-model, loop perbaikan, dan review bertahap. Kelebihannya bergantung pada konteks dan bukti yang dipakai, sementara konsensus tetap dapat salah. Penelitian ini diposisikan pada integrasi aturan keputusan yang dapat diaudit dengan deteksi, remediasi, dan pengujian ketahanan, disertai

pengukuran biaya. Pembeda tersebut masih merupakan hipotesis kontribusi yang perlu dibandingkan dengan implementasi pembanding, termasuk revisi SecAwareCoder, bukan klaim ketiadaan karya serupa [VERIFIKASI V04; VERIFIKASI V10].

2.4 Perbaikan Otomatis dan Pengendalian Regresi

2.4.1 Umpan balik eksternal untuk remediasi

Remediasi otomatis bertujuan menghilangkan penyebab kelemahan sambil mempertahankan perilaku yang dibutuhkan. Keberhasilan menghasilkan patch berbeda dari keberhasilan memperbaiki program: patch dapat menghapus alarm dengan membuang fungsi penting atau mengubah antarmuka. Karena itu, temuan awal, perubahan kode, serta hasil pemeriksaan setelah perubahan harus dihubungkan. Literatur perbaikan memberi dasar bagi loop revise, tetapi juga memperlihatkan bahwa ukuran keberhasilan sangat bergantung pada alat dan pengujian yang digunakan untuk menerima patch.

Feedback-Driven Security Patching menempatkan keluaran Bandit sebagai umpan balik eksternal untuk memperbaiki kode Python hasil LLM [Alrashedy et al., 2024]. Pendekatan ini berbeda dari meminta model mengkritik jawabannya tanpa observasi tambahan. Static Analysis as a Feedback Loop memperluas penggunaan diagnostik Bandit dan Pylint untuk beberapa dimensi kualitas kode [Blyth et al., 2025]. Sementara itu, LLMLOOP menghubungkan perbaikan sumber dan pengujian melalui sejumlah loop umpan balik [Ravi et al., 2026]. Ketiganya mendukung gagasan bahwa informasi terstruktur dari luar model membantu mengarahkan revisi, tetapi hilangnya diagnostik tertentu tetap harus dibedakan dari terpenuhinya kebutuhan program secara keseluruhan.

Penggabungan beberapa alat terlihat pada pendekatan Sriram et al. yang memakai retrieval, diagnostik GCC, CodeQL, dan eksekusi simbolik KLEE. Pada 1.720 program C/C++ dari CodeLlama-7B, tingkat cacat keamanan menurut CodeQL turun dari 58,55% menjadi 22,19%; bagian lain eksperimen memakai 1.522 program DeepSeek-Coder-1.3B [Sriram et al., 2026]. Batas domain ini penting: GCC dan KLEE tidak langsung menjadi konfigurasi pengujian untuk cuplikan

Python. Yang dapat diadopsi adalah prinsip menggabungkan sumber umpan balik dan memvalidasi ulang patch, bukan memindahkan angka penurunan atau perangkat uji tanpa penyesuaian.

2.4.2 Pemisahan Fixer dan penggunaan pengetahuan

Self-repair memungkinkan model pembuat kode menafsirkan umpan balik lalu memperbaiki keluarannya. Fixer yang terpisah memberikan kemungkinan sudut pandang berbeda, tetapi belum tentu lebih efektif atau independen. Model lain dapat mewarisi pola kelemahan serupa dan harus mempelajari kembali konteks tugas. Oleh karena itu, penggunaan model berbeda diusulkan sebagai variabel desain yang diuji terhadap model sama pada anggaran sebanding. Penelitian ini belum menentukan pasangan generator–fixer maupun jumlah revisi akhirnya [VERIFIKASI V09]. Klaim bahwa pemisahan peran pasti mengurangi bias tidak digunakan.

Retrieval dapat menyediakan contoh pola dan patch yang pernah berhasil. AutoPatch menggabungkan pengetahuan CVE dengan verifikasi serta patching, sedangkan pendekatan multi-tool menyimpan hasil revisi yang lolos pemeriksaan untuk digunakan kembali [Seo et al., 2025; Sriram et al., 2026]. Pada rancangan penelitian ini, Validation direncanakan menulis ke knowledge store yang dibaca Analyse, Triage, dan Fixer. Namun, keberhasilan lokal tidak boleh berubah menjadi label keamanan universal. Entri perlu mempertahankan konteks, versi kode, bukti uji, dan batas keberlakuannya. Pemisahan data evaluasi dari pengetahuan hasil eksperimen juga diperlukan agar perbaikan pada sampel uji tidak mencemari pengukuran berikutnya [VERIFIKASI V11].

2.4.3 Regresi, pemindaian ulang, dan penghentian

Pengujian dinamis dalam gate tidak selalu berarti pengujian aplikasi utuh. Untuk fungsi atau cuplikan, penelitian dapat membangun harness yang menyediakan masukan serta memeriksa kondisi kegagalan tertentu. Lingkungan terisolasi dan batas sumber daya direncanakan untuk menjalankan artefak tersebut. Namun, kemampuan harness merepresentasikan konteks asli harus diuji. Cuplikan yang bergantung pada layanan eksternal tidak dapat dianggap telah

diuji secara dinamis hanya karena lolos sintaks. Pemisahan antara penyusunan pengujian, pelaksanaan alat, dan penafsiran hasil menjadi tanggung jawab rancangan eksperimen.

Studi ESEM tentang perbaikan Infrastructure-as-Code menunjukkan bahwa perubahan iteratif dapat diikuti kegagalan pemeriksaan yang sebelumnya lolos. Penulis membedakan regresi yang tegas dari perubahan pengukuran akibat penataan ulang sumber daya [Agyekum dan Santos, 2026]. Temuan ini mengingatkan bahwa regresi perlu didefinisikan pada unit yang konsisten. Angka regresi Terraform tidak dipakai untuk memperkirakan risiko Python. Untuk gate yang diusulkan, regresi perlu mencakup munculnya temuan keamanan baru, gagalnya uji fungsional, serta perubahan perilaku yang menghilangkan kemampuan semula.

Setiap patch diusulkan kembali ke Sanitizer dan seluruh pemeriksaan yang berlaku. Pengujian yang sebelumnya memperlihatkan kelemahan dipertahankan sebagai uji regresi, sedangkan uji fungsional membantu memeriksa apakah perbaikan hanya menghindari tugas. Batas iterasi mencegah revisi berulang tanpa kemajuan, tetapi habisnya anggaran tidak mengubah artefak menjadi Verified. Kasus tersebut diarahkan sesuai kebijakan arbiter. Kelayakan pembuatan exploit atau fuzz harness oleh Testing LLM masih terbuka; ketika program belum dapat dieksekusi, hasil pengujian dicatat sebagai tidak tersedia, bukan lulus [VERIFIKASI V09; VERIFIKASI V12].

Sintesis dan Keterbatasan. Umpan balik alat dan retrieval mendukung remediasi, tetapi hasilnya bergantung pada domain, model, dan kriteria penerimaan. Pemisahan Fixer, pemindaian ulang, serta batas revisi layak diuji sebagai pengendalian proses. Evaluasi penelitian ini perlu melaporkan keberhasilan patch bersama regresi keamanan dan fungsionalitas. Dengan demikian, kualitas remediasi dinilai dari artefak setelah pemeriksaan ulang, bukan dari penjelasan model atau jumlah alarm yang berhasil dihilangkan saja.

2.5 Ketahanan Gate terhadap Manipulasi Reviewer

2.5.1 Permukaan ancaman dan perbedaan sasaran serangan

Reviewer LLM memproses teks yang dapat berasal dari pihak tidak tepercaya, termasuk komentar kode, docstring, keluaran alat, dan contoh dari knowledge store. *Prompt injection* terjadi ketika konten tersebut berusaha mengalihkan model dari tugas atau aturan penilaiannya. Pada gate, sasaran serangan dapat berupa penghilangan temuan, penurunan keparahan, atau pemberian keputusan pass yang tidak didukung bukti. Taksonomi ancaman agen menekankan bahwa pertahanan harus dinilai bersama kegunaan tugas dan konteks interaksi, bukan hanya kemampuan menolak instruksi mencurigakan [P. Wang et al., 2026].

Serangan terhadap generator, penilai umum, reviewer keamanan, dan editor yang dapat menjalankan perintah memiliki konsekuensi berbeda. TAPI menargetkan generasi kode melalui pemicu dalam komentar [Yang et al., 2024], sedangkan Your AI, My Shell membahas pengalihan editor agentik melalui sumber pengembangan yang diracuni [Liu et al., 2026]. Keduanya menunjukkan perlunya batas kepercayaan, tetapi keberhasilan serangannya tidak dapat dipindahkan langsung menjadi peluang mengelabui reviewer pada penelitian ini. Model ancaman perlu menetapkan bagian artefak yang boleh diubah penyerang, aksesnya terhadap aturan, serta kondisi yang dihitung sebagai keberhasilan [VERIFIKASI V13].

2.5.2 Bukti yang berbeda tentang kerentanan penilai

Studi LLM-as-a-Judge memperlihatkan bahwa penilaian model dapat dimanipulasi oleh injeksi instruksi dan membahas komite multi-model serta penilaian komparatif sebagai arah pertahanan [Maloyan dan Namiot, 2025]. Namun, tugas penilaian umum tidak identik dengan pemeriksaan kerentanan kode. Komite juga dapat menerima konten penyerang yang sama. Oleh sebab itu, rekomendasi tersebut merupakan dasar rancangan eksperimen, bukan bukti bahwa menambah reviewer akan menurunkan keberhasilan serangan pada semua kondisi.

Hasil Thornton memberikan pembanding yang penting. Pada 100 sampel rentan Python, JavaScript, dan Java yang dinilai delapan model komersial maupun terbuka, efek agregat komentar adversarial tidak signifikan secara statistik pada pengujian berpasangan. Penghapusan komentar justru menurunkan deteksi pada model yang lebih lemah, sementara penggunaan temuan SAST sebagai petunjuk membantu penilaian [Thornton, 2026]. Temuan tersebut tidak membuktikan kekebalan terhadap semua prompt injection. Datasetnya berfokus pada kode rentan dan serangan komentar tertentu, sehingga kemampuan mengenali kode tidak rentan serta vektor lain memerlukan pengujian tambahan.

2.5.3 Sanitizer dan batas kebijakan sebagai hipotesis pertahanan

Pengujian serangan perlu mempertahankan kelemahan program yang hendak disembunyikan. Jika transformasi mengubah perilaku atau menghilangkan kerentanannya, perubahan keputusan reviewer tidak lagi mengukur manipulasi penilaian pada kasus yang sama. Skenario komentar yang mempertahankan kode dapat dipisahkan dari skenario yang mengubah string atau struktur program. Pasangan sampel dan label diperiksa sebelum evaluasi, sementara tingkat kesulitan kerentanan dasar dicatat terpisah dari keberhasilan serangan. Langkah ini merupakan usulan kontrol eksperimen agar peningkatan ketahanan tidak tertukar dengan perubahan objek yang dinilai.

Sanitizer non-LLM direncanakan membuat representasi masukan reviewer yang mengurangi komentar, docstring, atau instruksi tersembunyi. Penghapusan harus diperlakukan sebagai transformasi yang diuji, bukan langkah yang pasti mempertahankan konteks. Dalam Python, docstring dapat diakses saat eksekusi dan string biasa dapat menjadi bagian perilaku program; komentar juga dapat memuat petunjuk alat. Karena itu, usulan implementasinya perlu memisahkan proyeksi untuk reviewer dari artefak asli yang dipindai dan dijalankan, sambil memelihara pemetaan lokasi. Keutuhan perilaku serta dampaknya pada deteksi harus divalidasi [VERIFIKASI V14].

Ablasi diusulkan membandingkan gate tanpa sanitizer dan gate dengan sanitizer pada sampel identik, baik bersih maupun diserang. Perubahan presisi,

recall, penerimaan kode rentan, dan kegunaan perlu dibaca bersama. Serangan yang sebelumnya gagal karena kerentanan telah dikenali tidak boleh disamakan dengan kelemahan yang memang tidak pernah terdeteksi. *Attack success rate* karenanya perlu menyebut penyebutnya, termasuk apakah dihitung hanya pada kasus yang semula berhasil ditolak. Jika sanitizer tidak membantu atau merugikan, hasil itu tetap relevan untuk menentukan batas pertahanan yang layak.

Arbiter berjangkar SAST membatasi kemampuan reviewer menghapus temuan penting, tetapi bukan pengganti perlindungan seluruh alur. PhantomSkill menunjukkan bahwa perilaku berbahaya dapat disamarkan sebagai implementasi yang tampak memiliki kelemahan biasa [Lin dan Yu, 2026]. Konten manipulatif juga dapat berada pada pengenal, string, diagnostik, atau pengetahuan yang diambil kembali. Aturan YAML, catatan alat, dan jalur penulisan knowledge store perlu dipisahkan dari kewenangan penyerang. Verifikasi dependensi dan *slopsquatting* belum menjadi komponen ruang lingkup; ancaman itu dicatat sebagai batas cakupan, tanpa menganggap gate telah menanganinya [VERIFIKASI V11; VERIFIKASI V13].

Sintesis dan Keterbatasan. Literatur mendukung adanya risiko manipulasi, tetapi tingkat kerentanannya bergantung pada tugas, model, dan vektor serangan. Bukti yang berbeda menuntut evaluasi berpasangan serta kontrol kegunaan. Posisi penelitian ini adalah mengukur pengaruh sanitizer dan kebijakan keputusan pada gate secara keseluruhan, dengan kemungkinan hasil positif, kecil, maupun negatif. Tidak digunakan klaim bahwa pertahanan komentar menyelesaikan seluruh prompt injection atau bahwa reviewer LLM selalu mudah dikelabui.

2.6 Ringkasan Penelitian Terdahulu dan Celah Penelitian

2.6.1 Perbandingan cakupan dan bukti

Penelitian terdahulu menunjukkan pergeseran dari pengukuran kelemahan keluaran menuju penggabungan alat, koordinasi agen, dan perbaikan iteratif. Tabel berikut membandingkan pendekatan berdasarkan objek evaluasi dan batas kesimpulannya. Angka hanya dicantumkan jika konteks utamanya

telah diperiksa. Keterangan bahwa suatu aspek belum dievaluasi dibatasi pada bagian sumber yang ditinjau, bukan pernyataan bahwa kemampuan tersebut mustahil tersedia dalam artefak lain. Baris yang memuat tanda verifikasi masih membutuhkan pemeriksaan sebelum digunakan sebagai perbandingan final.

Tabel 2: Perbandingan penelitian terdahulu untuk multi-agent security gate

Studi	**Pendekatan**	**Dataset / objek**	**Hasil utama**	**Batas perbandingan dengan gate**
AutoSafeCoder; Nunez et al. (2024)	Coding, reviewer LLM, fuzzing; iteratif	SecurityEval; HumanEval; GPT-4o	Kode rentan 59/121 menjadi 44/121; pass@1 sedikit turun	Reviewer statis berbasis LLM; Bandit sebagai penilai eksperimen. Ketahanan reviewer belum menjadi evaluasi utama.
Teaming-Up LLMs; Sabir et al. (2026)	Ensembel, CodeQL, verifikasi dan patching lintas-model	SecLLMEval dan SecLLMHolmes; C/Python	Hibrida dilaporkan lebih konsisten dari kolaborasi LLM saja	Sudah mencakup alur ujung ke ujung. Angka antarkonfigurasi perlu rekonsiliasi [VERIFIKASI V05].
ReasonVul; Peng et al. (2026)	Tiga gaya penalaran dan debat	PrimeVul; JITVUL; model heterogen	389/542 konflik diselesaikan benar pada analisis PrimeVul	Fokus deteksi pasangan kode, bukan keputusan penerimaan dan remediasi pada gate generasi.
VulTrial; Widyasari et al. (2025)	Peran ruang sidang dan penyesuaian instruksi	Benchmark deteksi; antara lain GPT-4o	Perbaikan deteksi pada konfigurasi penulis	Peran hakim/juri tidak identik dengan arbiter kebijakan pass/revise/ block; rincian artefak perlu dibandingkan.
MAVUL; Y. Li et al. (2025)	Penalaran kontekstual dan penyempurnaan lintas-peran	Dataset kerentanan berpasangan	Peningkatan dilaporkan; angka rinci tidak dikutip	Metadata dan abstrak primer diperiksa; konfigurasi dan ukuran efek perlu audit [VERIFIKASI V10].
QASecClaw; Ameen et al. (2026)	Semgrep, filter LLM, korelasi bukti	OWASP v1.2; 2.740 kasus Java; Qwen 3.5 Plus	False positive 560 menjadi 64	Recall ikut turun; jalur dinamis tidak aktif pada evaluasi. Fokus penyaringan temuan, bukan generasi dan fixer.
SAST-Genius; Agrawal dan Ahi (2025)	Semgrep dan analisis LLM	Pengujian dalam laporan penulis	False positive dilaporkan berkurang	Rincian benchmark dan replikasi angka belum diaudit [VERIFIKASI V06].
ZeroFalse; Iranmanesh et al. (2025)	Konteks aliran dan prompt khusus CWE	OWASP Java Benchmark; OpenVuln	Keseimbangan presisi–recall dilaporkan membaik	Memvalidasi alarm pada kode tersedia; pemindahan ke generasi Python memerlukan evaluasi.
IRIS; Z. Li et al. (2025)	Inferensi taint LLM dan CodeQL	120 kerentanan CWE-Bench-Java; GPT-4	55 terdeteksi dibanding 27 pada CodeQL	Analisis proyek Java; bukan bukti langsung efektivitas pada cuplikan Python hasil LLM.
FDSP; Alrashedy et al. (2024)	Umpan balik Bandit untuk patching	PythonSecurity Eval; kode Python hasil LLM	Umpan balik eksternal mendukung perbaikan	Tidak diperlakukan sebagai pembuktian keunggulan fixer heterogen; keberhasilan perlu uji ulang independen.
RAG + Multi-Tool; Sriram et al. (2026)	Retrieval, GCC, CodeQL, KLEE	3.242 program C/ C++; dua model	Pada CodeLlama-7B, cacat keamanan 58,55% menjadi 22,19%	Self-repair dan tugas C/ C++ pendek; tidak langsung merepresentasikan pengujian Python.

Studi	Pendekatan	Dataset / objek	Hasil utama	Batas perbandingan dengan gate
Maloyan dan Namiot (2025)	Injeksi instruksi terhadap penilai LLM	Beberapa tugas penilaian	Menunjukkan manipulasi penilaian pada protokol penulis	Bukan estimasi langsung keberhasilan serangan pada reviewer keamanan atau gate ini.
Thornton (2026)	Serangan komentar dan evaluasi pertahanan	100 sampel rentan; tiga bahasa; delapan model	Efek komentar tidak signifikan pada pengujian berpasangan	Comment stripping dapat merugikan; cakupan sampel rentan tidak cukup untuk menilai false positive.
CodeSecEval / SecAwareCoder; J. Wang et al. (2026)	Pemodelan ancaman dan umpan balik eksekusi	v3: 255 tugas Python; 77 CWE	Revisi memperluas benchmark serta kerangka generasi	Pembanding tambahan dari revisi sumber yang sama; rincian sistem dan uji ketahanannya perlu dibaca [VERIFIKASI V04].

2.6.2 Celah integrasi, pertahanan, dan pengukuran

Celah tersebut dibatasi oleh sifat penelusuran yang belum merupakan tinjauan sistematis menyeluruh. Sumber yang baru diperiksa abstraknya memberikan petunjuk cakupan, tetapi belum cukup untuk menyatakan bahwa suatu komponen tidak tersedia. Sebaliknya, arsitektur yang mencantumkan pengujian dinamis belum tentu mengevaluasi jalur tersebut. Pembedaan ini dipertahankan dalam tabel dan daftar verifikasi. Sebelum klaim kontribusi difinalkan, penulis perlu memeriksa kode, konfigurasi, dan protokol pembanding terdekat. Pengujian ulang pada subset bersama akan lebih informatif daripada membandingkan angka tertinggi dari masing-masing paper. Dengan cara itu, posisi penelitian ditentukan melalui pertanyaan yang dapat diuji, yaitu pengaruh kebijakan keputusan dan pertahanan masukan terhadap kualitas keluaran, serta biaya tambahan yang harus dibayar untuk memperoleh pengaruh tersebut.

Terdapat tiga hubungan penting antartema. Pertama, deteksi dan remediasi membutuhkan label serta bukti yang konsisten. Triase yang menghapus alarm tidak identik dengan perbaikan program; patch yang lolos pemindai belum tentu mempertahankan fungsi. Kedua, penambahan agen tidak menghilangkan kebutuhan pengujian empiris karena kesalahan dapat berkorelasi atau diperkuat melalui konsensus. Ketiga, pertahanan reviewer harus diuji bersama dampaknya pada tugas utama. Sanitizer yang menolak manipulasi tetapi sekaligus menghapus konteks relevan dapat menurunkan kegunaan gate. Hubungan ini mengarahkan evaluasi pada keputusan akhir artefak, bukan hanya kualitas jawaban setiap agen.

Rangkaian generasi, pemindaian, review, dan perbaikan telah memiliki pembanding langsung, terutama AutoSafeCoder, Multi-LLMSecCodeEval, serta AutoPatch. Triase dan korelasi bukti juga dekat dengan QASecClaw, sementara review dengan gerbang bertahap telah dibahas Refute-or-Promote. Revisi SecAwareCoder memperkuat alasan untuk memeriksa literatur mutakhir sebelum menetapkan pembeda. Karena itu, bentuk dasar pipeline dan keberadaan arbiter tidak diklaim sebagai kebaruan tersendiri. Ruang kontribusi yang diajukan berada pada integrasi batas keputusan, pengujian pertahanan masukan reviewer, pemisahan fixer, dan pelaporan perubahan keamanan, fungsionalitas, serta biaya pada protokol yang sama.

Rancangan yang ditinjau dalam penelitian ini menghubungkan generator dengan Sanitizer, Analyse berbantuan SAST, Triage, Testing, Validation, dan arbiter berbasis aturan. Fixer direncanakan menghasilkan patch ketika keputusan revise diberikan, lalu artefak diperiksa kembali. Knowledge store hanya direncanakan menerima pola dan patch yang telah melalui Validation. Peran Testing LLM, kelayakan pengujian dinamis pada cuplikan, pasangan model, dan batas iterasi belum ditetapkan. Penyebutan komponen tersebut menunjukkan arah integrasi, bukan hasil implementasi atau bukti efektivitas [VERIFIKASI V09; VERIFIKASI V11; VERIFIKASI V12].

Evaluasi diusulkan mengukur precision, recall, dan F1 per CWE, serta tingkat keluaran rentan yang diterima, keberhasilan patch, dan regresi fungsional. Pengukuran ketahanan menggunakan pasangan kode bersih dan kode yang diserang dengan definisi keberhasilan yang eksplisit. Konsumsi token dan latensi dihitung sepanjang alur, termasuk pengulangan dan kegagalan. Baseline LLM tunggal, SAST saja, serta SAST ditambah LLM dilengkapi ablasi sanitizer, aturan jangkar, dan fixer heterogen apabila anggaran memungkinkan. Penilaian akhir memerlukan label atau uji yang tidak hanya menyalin keputusan alat di dalam gate agar keuntungan tidak berasal dari penggunaan alat ukur yang sama [VERIFIKASI V15].

Sintesis dan Keterbatasan. Penelitian ini mengusulkan integrasi pemeriksaan, remediasi, dan aturan penerimaan dalam satu gate, kemudian mengevaluasinya terhadap efektivitas, serangan reviewer, serta biaya. Pada penelusuran terbatas ini, kombinasi pengukuran tersebut masih memerlukan perbandingan langsung dengan artefak studi terdekat; kesimpulan kebaruannya bersifat sementara. Ruang lingkup Python dan lima sampai delapan CWE membatasi generalisasi. Hasil yang tidak menunjukkan peningkatan tetap bermakna apabila protokol, ketidakpastian, dan biaya dilaporkan secara terbuka.

BAB III

ANALISIS DAN PERANCANGAN SOLUSI

3.1 Analisis Masalah

Bab ini menguraikan hasil analisis yang lebih mendalam tentang persoalan tugas akhir yang diangkat.

3.2 Kebutuhan Sistem

Kebutuhan sistem mencakup kebutuhan fungsional dan non-fungsional:

1. **Kebutuhan Fungsional (F)**: Fungsionalitas inti yang harus disediakan oleh perangkat lunak.
2. **Kebutuhan Non-Fungsional (NF)**: Kriteria kualitas sistem seperti performa, keandalan, dan keamanan.

3.3 Perancangan Arsitektur

Jelaskan perancangan arsitektur sistem secara menyeluruh, aliran data, serta modul-modul utama yang dibangun.

BAB IV
IMPLEMENTASI DAN EVALUASI

4.1 Lingkungan Implementasi

Jelaskan spesifikasi lingkungan perangkat keras, perangkat lunak, pustaka, dan kakas yang digunakan dalam pengembangan sistem.

4.2 Prosedur dan Skenario Pengujian

Jelaskan bagaimana pengujian atau eksperimen dirancang untuk mengukur pencapaian tujuan tugas akhir.

4.3 Hasil Pengujian dan Diskusi

Tampilkan data-data hasil evaluasi beserta analisis pembanding terhadap solusi pembanding (*baseline*).

BAB V
KESIMPULAN DAN SARAN

5.1 Kesimpulan

Kesimpulan menjawab setiap poin tujuan tugas akhir yang telah dinyatakan pada BAB I berdasarkan temuan empiris pada BAB IV.

5.2 Saran

Saran memuat rekomendasi konkret untuk pengembangan atau penelitian lanjutan di masa mendatang.

DAFTAR PUSTAKA

LAMPIRAN A

PERNYATAAN PENGGUNAAN KECERDASAN BUATAN (AI)

Dengan ini, saya:

Nama Mahasiswa : Muhammad Adha Ridwan
NIM : 13523098
Program Studi : Sarjana Teknik Informatika
Fakultas / Sekolah : Sekolah Teknik Elektro dan Informatika
Judul Tugas Akhir : <JUDUL TUGAS AKHIR>

Menyatakan bahwa dalam pengerjaan dan penulisan Tugas Akhir ini, saya **menggunakan / tidak menggunakan** bantuan perangkat kecerdasan buatan (AI). Rincian penggunaan AI yang dilakukan adalah sebagai berikut:

Tabel 3: Rincian Penggunaan Alat AI dalam Penyusunan Tugas Akhir

Lingkup Pekerjaan	Digunakan?	Rincian dan Alat yang Digunakan
Pemeriksaan Tata Bahasa & Ejaan	Ya/Tidak	Perbaikan ejaan teks dokumen (misal: Grammarly, DeepL)
Bantuan Diskusi Konseptual	Ya/Tidak	Diskusi awal pemahaman algoritma (misal: LLM)
Pembuatan Grafis atau Diagram	Ya/Tidak	Pembuatan diagram alur data atau skema arsitektur
Penulisan Kode Sumber Program	Ya/Tidak	Pencarian referensi pustaka dan debugging fungsi spesifik

Saya menyatakan bertanggung jawab penuh atas seluruh kebenaran fakta, analisis, kode program, dan kesimpulan yang disajikan dalam Laporan Tugas Akhir ini.

Bandung, <Tanggal Pengesahan>

Muhammad Adha Ridwan

NIM : 13523098

LAMPIRAN B
CONTOH LAMPIRAN TEKNIS

B.1 Spesifikasi Lingkungan Pengujian

Uraikan informasi tambahan yang terlalu panjang atau detail jika diletakkan pada bagian utama naskah, seperti spesifikasi perangkat keras, konfigurasi server, atau cuplikan kode skrip evaluasi.